Rigidity of the Rank-48 4×4 Matrix-Multiplication Scheme under Solved Flip Moves — a Machine-Checked Obstruction

Greg Sidebottom

Research note — logic repo (https://github.com/gsidebottom/logic), matmul track, 2026-07-08. Companion to "A 55-Addition Rank-23 Scheme for 3×3 *Matrix Multiplication via Exact Two-Sided Minimization" (`doc/matmul_adds_paper.md`, artifacts DOI `10.5281/zenodo.21240904`); every claim here has a mechanical check (§7), and the main theorem is machine-checked in Lean 4 (§5).*

Abstract

Fast matrix multiplication of 4×4 matrices currently stands at **48 multiplications** over the real numbers — a scheme found by AlphaEvolve (2025) over the complex numbers and given rational coefficients by Dumas, Pernet, and Sedoglavic (2025). Whether 47 is possible outside characteristic 2 is a central open problem, and the most productive discovery machinery of recent years — **flip graphs** (Kauers–Moosbauer 2022) — has never produced a characteristic-0 improvement at this format despite sustained effort.

This note offers a certified explanation. We generalize flip moves from their native mod-2 setting to the rationals, where the flip scalar becomes a free parameter λ and two new phenomena appear: exact factor coincidences — the events that enable rank reductions — occur with probability zero under random moves, so productive moves must be **solved for** (a two-unknown linear system per candidate); and solvable "coincidence" moves turn out to be abundant. We then prove, by exhaustive enumeration in exact arithmetic:

> **Theorem (rigidity).** From the rational rank-48 scheme, the graph generated by one *split* (any pair of summands, any tensor slot) followed by arbitrary sequences of *solved flips* closes on a finite component of exactly **7,408 states**, every state of which is a verified decomposition of the 4×4 multiplication tensor, and **no state admits a reduction below 49 summands except the trivial return to the seed**. In particular, no rank-47 scheme — which would be a characteristic-0 record — is reachable this way.

The theorem is verified end to end in **Lean 4** by reflection (`native_decide`), with deduplication on full canonical forms (no hashes), unbounded exact integers (no magnitude caps), saturation of the search asserted *inside* the decided proposition, and the seed self-certified against the definition of the multiplication tensor.

1 Background, with definitions

Bilinear schemes and rank. Multiplying two $n \times n$ matrices is a bilinear map. A *bilinear scheme* (equivalently, a rank-r decomposition of the multiplication tensor $\langle n, n, n \rangle$) computes it as

$$M_t = \Big(\sum_{i,j} \alpha_t[i,j]\, A[i,j]\Big) \cdot \Big(\sum_{k,l} \beta_t[k,l]\, B[k,l]\Big), \quad t = 1..r, \qquad C[p,q] = \sum_t \gamma_t[p,q]\, M_t$$

for coefficient vectors $(\alpha_t, \beta_t, \gamma_t)$. Each product M_t is a *rank-one summand* $a_t \otimes b_t \otimes c_t$ of the tensor; r is the scheme's **rank** — the number of (expensive, non-commutative) multiplications. Validity is a system of polynomial constraints on the coefficients, the **Brent equations** (Brent 1970): for $\langle 4, 4, 4 \rangle$ there are $16^3 = 4096$ of them. Because the products never commute A- and B-entries, a valid scheme works over any ring — in particular over *block matrices*, which is what makes low rank matter: a rank-r scheme for $\langle 4, 4, 4 \rangle$ yields an $O(n^{\log_4 r})$ algorithm by recursion.

Characteristic. The *characteristic* of a field is the smallest number of times 1 must be added to itself to reach 0 — e.g. 2 for the Boolean field $\mathrm{GF}(2) = \{0, 1\}$ (where $1 + 1 = 0$), and *characteristic 0* for $\mathbb{Q}, \mathbb{R}, \mathbb{C}$, where no repeated sum of 1's vanishes. The distinction is essential here: a scheme with coefficients in $\mathrm{GF}(2)$ computes matrix products *only* for matrices over fields of characteristic 2. AlphaTensor's celebrated rank-47 scheme for $\langle 4, 4, 4\rangle$ (Fawzi et al. 2022) is of this kind — it does **not** give a 47-multiplication algorithm for real matrices. For real (or floating-point, or integer) linear algebra one needs characteristic-0 coefficients, and there the record is:

rank	scheme	coefficients
49	Strassen 1969, squared (2×2 scheme applied twice)	± 1
48	AlphaEvolve (Novikov et al. 2025)	$\mathbb{C}$ (halves of Gaussian integers)
48	Dumas–Pernet–Sedoglavic (DPS) rational point of the same scheme	$\mathbb{Z}[\frac{1}{2}]$
47	**open** in characteristic 0	—

Two structural facts sharpen the picture (Moran–Schwartz–Yuan 2026): the AlphaEvolve scheme admits **no integer-coefficient equivalent** (the halves are intrinsic), and the only other known rank-48 scheme — Kaporin's (2024), found independently by nonlinear least squares — admits **no real-coefficient equivalent at all** (its orbit requires cube roots of 2). So the DPS rational scheme is, up to the natural symmetries, *the* rank-48 object available to characteristic-0 computation, and the question "can flip methods improve 4×4 over the reals?" is a question about this one scheme. We call it **the seed** throughout.

Equivalence (de Groote symmetry). Two schemes are *equivalent* if one maps to the other under the isotropy group of the multiplication tensor — invertible "sandwiching" of the three coefficient families plus permutations of the three tensor slots (de Groote 1978). All statements about "the" scheme are statements about its orbit.

Flip graphs. Kauers and Moosbauer (2022) organize all rank-r schemes into a graph. Two summands that share a factor in one slot, say $a_i = a_j = a$, admit a **flip**:

$$a \otimes b_i \otimes c_i + a \otimes b_j \otimes c_j \;=\; a \otimes (b_i{+}b_j) \otimes c_i + a \otimes b_j \otimes (c_j{-}c_i)$$

— an exchange of material that preserves the sum (hence validity) and the rank. Two summands proportional in *two* slots merge into one — a **reduction**, decreasing the rank. A **split** (Moosbauer–Poole 2025) is the reverse of a reduction: replace a summand by two whose chosen-slot factors sum to it, *increasing* the rank to escape local structure. Random walks over flips punctuated by reductions found new records in characteristic 2 (among them 5×5 in 93 multiplications), making flip graphs the most productive scheme-discovery machinery outside large-scale machine learning.

The failures this note explains. At the 4×4 format the flip approach has conspicuously stalled, in ways we and others have measured:

- In characteristic 2, the two known rank-47 schemes (AlphaTensor's and the flip-graph one) sit in near-isolated flip-graph components; our earlier experiments (engine `src/flip.rs` in the companion repository) additionally verified that **neither lifts to ± 1 integer coefficients** — so neither yields a characteristic-0 algorithm — and that mod-2 flip descent from random rank-49 starting points stalls: our probe of ∼100,000 descent runs found every rank-49 landing point to be **100% flip-isolated** (no two summands sharing any factor — a flip-graph sink).
- In characteristic 0 the machinery could not even be applied to the one interesting seed: the DPS scheme's coefficients contain $\frac{1}{2}$'s, so it **has no mod-2 reduction at all**, and the published flip tooling is mod-p.

This note builds the missing rational tooling, measures what the rational flip landscape around the seed actually looks like, and proves that the natural search cannot leave it.

2 Rational flip moves

Everything below is implemented in `src/bin/flip48.rs` (Rust), with exact arithmetic throughout: a factor vector is stored as sixteen integers times a power of two (the coefficient ring $\mathbb{Z}[\frac{1}{2}]$ of the seed is closed under all moves), and every walk state is verifiable against the full Brent system in 128-bit exact integer arithmetic.

Canonical gauge. Each summand $a \otimes b \otimes c$ carries a projective freedom $(a, b, c) \to (ua,\ vb,\ (uv)^{-1}c)$. We fix it by keeping a and b *primitive* — integer entries with no common factor, leading entry positive — with all scalar content folded into c. Under this gauge, proportionality of a- or b-factors is literal equality, which is what the move machinery tests. (An early version of the engine canonicalized only powers of two, making proportionality by odd factors invisible — a completeness bug worth recording, since walks create factors of 3 immediately.)

λ-flips. Over a field the flip identity holds with any scalar:

$$a \otimes b_i \otimes c_i + a \otimes b_j \otimes c_j = a \otimes (b_i + \lambda b_j) \otimes c_i + a \otimes b_j \otimes (c_j - \lambda c_i).$$

Over GF(2) the only choice is $\lambda = 1$; over $\mathbb{Q}$ the move is a one-parameter family. The engine restricts λ to $\pm 2^k$ (dyadic scalars) so that all coefficients remain in $\mathbb{Z}[\frac{1}{2}]$.

Reductions on any two slots. Summands proportional in slots (a, b), (a, c), or (b, c) merge on the third slot, with the proportionality ratios folded in exactly.

Universal splits. Over $\mathbb{Q}$, splits acquire a power they lack mod 2: *any* summand i can be split against *any* other summand k in any slot — write $a_i = \mu a_k + (a_i - \mu a_k)$ — manufacturing a shared factor with a partner of our choosing. Mobility can be engineered; it need not be found.

3 What the rational flip landscape looks like

Three measurements, all exact, set up the theorem.

(1) The seed is flip-isolated. Its 48 summands are pairwise non-proportional in *every* slot: not a single classical flip is available. This is the same signature as the characteristic-2 rank-47 schemes — the interesting objects at this format all sit at flip-graph sinks. Splits are therefore not an optimization but the only way to move at all.

(2) Over $\mathbb{Q}$, coincidences never happen by chance. A reduction requires two summands to become *exactly* proportional in two slots. Mod 2 the state space is finite and random walks hit such coincidences constantly; over $\mathbb{Q}$ the walk wanders an infinite space in which exact coincidence is a measure-zero event. Empirically: ten million random-λ moves from the seed produced **zero** coincidences. Every reduction ever observed was a split un-doing itself. Random walking — the engine of all published flip-graph results — is the wrong paradigm for characteristic 0.

(3) Solvable coincidences are abundant. The cure is to *solve* for the moves. For a pair (i, j) sharing slot s and a third summand m, the requirement "after a flip with scalar λ, factor f_i becomes proportional to f_m" is the 16-equation, 2-unknown linear system

$$f_i + \lambda f_j = \mu f_m,$$

solvable exactly by Cramer's rule; the engine accepts solutions with dyadic λ. Such *coplanar triples* turn out to be plentiful: around 3.5 million solved flips were applied in a 45-second, 12-thread run. The search space is rich — which makes what follows meaningful.

4 The rigidity theorem

With mobility engineered by splits and productive moves solved for, the reachable landscape can be enumerated outright — no randomness.

Enumeration. For every ordered pair of summands (i, k) and every slot s — $48 \cdot 47 \cdot 3 = 6768$ roots — split i against k in s; then explore by depth-first search: at each state, find every shared-factor pair, compute every solved flip, apply each, and deduplicate states; at every state, run the reduction sweep and record the outcome. The split scalar μ does not appear in any coincidence system (the split part inherits the other two factors of the split summand unchanged), so taking $\mu = 1$ loses no generality for reachability of coincidences.

Result. The exploration **saturates**: the union of all components closes after ∼8500 node visits (7,408 distinct states under global deduplication), stable from depth 4 onward and identical under magnitude caps 256 and 1024 (the component never approaches either). Within it:

- every state is a valid rank-49 or rank-48 decomposition (verified against all 4096 Brent equations, exactly);
- **every reduction sweep terminates at ≥ 49 summands, or at 48 by the trivial merge that returns the original seed**;
- in particular, **no rank-47 state exists in the component**, and no rank-48 state other than the seed — the solved-move system cannot even produce a *different* presentation of rank 48, let alone a smaller one.

We call this property **rigidity of the seed under solved moves**. The scope is stated precisely: one split (all pairs, all slots, all dyadic scalars via the μ-independence above), then arbitrary sequences of solved flips with dyadic λ; shared factors in the two canonical slots are detected up to proportionality, in the scalar-carrying slot by exact equality. What lies outside the scope — and outside any current method's reach — is discussed in §6.

5 The machine-checked proof

Exhaustive computational claims deserve mechanical verification; the enumeration above is small enough to re-run inside a proof assistant. The Lean 4 development `matmul/mm55proof/Mm55proof/Rigid48.lean` (same Lake project as the 3×3 correctness proof of the companion paper) re-implements the entire move system — gauge, λ-flips, splits, all three reduction patterns, the Cramer solver, the DFS — as *total* executable functions over Lean's unbounded integers, and proves:

```
theorem seed_valid : checkDecomp seed = true
theorem rigid : explore seed 20000 = <7408, true, true, true>
```

by reflection (`native_decide`): the enumeration runs inside Lean's native evaluator and the kernel accepts the resulting boolean. The four components of the pinned tuple assert, in order: the component has exactly 7,408 states; the search *saturated* (fuel was not exhausted — closure is part of the theorem, not an external observation); every visited state passes the full Brent check; and no reduction sweep lands below 49 summands except at the seed itself.

The formalization is stronger than the Rust run in four ways:

1. **collision-free deduplication** — visited states are keyed by their full canonical forms, not 64-bit hashes;
2. **no magnitude caps** — Lean's arbitrary-precision integers remove the caps entirely from the statement;
3. **saturation inside the decided proposition**;
4. **a self-certifying seed** — the 48-summand literal (exported by `flip48 --emit-lean`) is proved to decompose the *defined* multiplication tensor, so no trust in the exporter is required.

Trust base: Lean's standard axioms plus the `native_decide` compiler-trust axiom (printed by the build's `#print axioms` audit). The component count was first learned from a compiled probe executable and only then pinned into the theorem — a wrong pin fails the build, guarding against a vacuous pass. `lake exe rigid48probe` re-runs the exploration natively in about $3\frac{1}{2}$ minutes.

6 What rigidity does and does not mean

It explains the observed failures. The one rank-48 scheme available to characteristic-0 computation sits at a flip sink; random rational walks provably cannot generate coincidences; and the systematic solved-move search closes on a small component containing no exit. Every mechanism by which flip methods have ever found records is individually and jointly obstructed at this seed — consistent with three years of silence at 4×4 in characteristic 0, and with our own measured stalls in characteristic 2.

It is scoped, honestly. Rigidity is proved for the solved-move system: single splits, dyadic λ, solved flips. Conceivable escapes outside the scope: multi-split states (rank 50+ before descending); non-dyadic rational λ (leaving $\mathbb{Z}[\frac{1}{2}]$; the seed's own ring makes this unnatural but not impossible); moves through *unsolved* territory followed by later solves (the measure-zero argument makes this unpromising but it is heuristic, not proven); and entirely different seeds — though the classification results quoted in §1 say there are none over $\mathbb{R}$ at rank 48 today. Extending the certificate to two-split radii is a finite (larger) enumeration of the same kind.

A structural reading. The Moran–Schwartz–Yuan results show the scheme's $\frac{1}{2}$-coefficients are intrinsic; our measurements show its factor geometry is maximally spread (48 distinct directions in every slot, coplanarities present but never twice-aligned). Rigidity adds: the scheme is not merely isolated but *locally unique* — its neighborhood in the solved-move graph contains no second rank-48 point. Discovery of a characteristic-0 rank-47, if one exists, will require machinery that does not move through this landscape — large-scale learned search (how the 48 itself was found), algebraic construction, or SAT-style exhaustive methods at presently inaccessible scales.

7 Reproduction

Prerequisites: a Rust toolchain; `elan` for the Lean part (toolchain and Mathlib commit pinned in the repository).

```
# the engine: exact seed verification + the walk/enumeration findings
cargo build --release --bin flip48
./target/release/flip48 --seconds 45 --threads 12      # walk stats
./target/release/flip48 --pursue3 10 --cap 256         # saturation
./target/release/flip48 --pursue3 10 --cap 1024        # cap-independence

# the machine-checked theorem
cd matmul/mm55proof
lake exe cache get     # prebuilt Mathlib
lake build             # elaborates rigid + seed_valid (native_decide;
                       # the Rigid48 module takes ~15 minutes)
lake exe rigid48probe  # re-runs the exploration, prints
                       # visited=7408 saturated=true allValid=true noBad=true
```

The seed's provenance: `matmul/dps48/{L,R,P}.sms` are the Dumas–Pernet–Sedoglavic matrices fetched from their PLinOpt repository; `matmul/probe48.py verify` proves them against all 4096 Brent equations over exact rationals; `flip48 --emit-lean` exports the gauged seed, which Lean re-certifies independently (`seed_valid`).

Acknowledgments

This work was carried out in an extended interactive collaboration with Claude (Anthropic; the Fable 5 and Opus 4.8 models), which implemented the search engines, the Lean formalization, and much of this text under the author's direction and review. Every computational claim is mechanically checkable by the commands in §7; the results rest on those independent verifications rather than on trust in either the author or the tools.

References

- V. Strassen. *Gaussian Elimination is not Optimal.* Numerische Mathematik 13:354–356 (1969).
- R. P. Brent. *Algorithms for Matrix Multiplication.* Stanford report CS-TR-70-157 (1970).
- H. F. de Groote. *On Varieties of Optimal Algorithms for the Computation of Bilinear Mappings I/II.* Theoretical Computer Science 7 (1978).
- A. Fawzi, M. Balog, A. Huang, T. Hubert, B. Romera-Paredes, M. Barekatain, A. Novikov, F. J. R. Ruiz, J. Schrittwieser, G. Swirszcz, D. Silver, D. Hassabis, P. Kohli. *Discovering Faster Matrix Multiplication Algorithms with Reinforcement Learning* (AlphaTensor). Nature 610:47–53 (2022).
- M. Kauers, J. Moosbauer. *Flip Graphs for Matrix Multiplication.* arXiv:2212.01175 (2022).
- J. Moosbauer, M. Poole. *Flip-Graph Algorithms for Matrix Multiplication in Larger Formats* (and successors); see also M. Kauers, I. Wood, *Exploring the Meta Flip Graph for Matrix Multiplication*, arXiv:2510.19787 (2025).
- A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, M. Balog. *AlphaEvolve: a Coding Agent for Scientific and Algorithmic Discovery.* arXiv:2506.13131 (2025).
- J.-G. Dumas, C. Pernet, A. Sedoglavic. *A Non-Commutative Algorithm for Multiplying* 4×4 *Matrices Using 48 Non-Complex Multiplications.* arXiv:2506.13242 (2025). Artifacts in the PLinOpt repository (`github.com/jgdumas/plinopt`).
- J.-G. Dumas, C. Pernet, A. Sedoglavic. *A More Accurate Rational Non-Commutative Algorithm for Multiplying* 4×4 *Matrices Using 48 Multiplications.* arXiv:2603.18699 (2026).
- I. Kaporin. *Finding Complex-Valued Solutions of Brent Equations Using Nonlinear Least Squares.* Computational Mathematics and Mathematical Physics 64(9):1881–1891 (2024).
- Y. Moran, O. Schwartz, S. Yuan. *Complex to Rational Fast Matrix Multiplication.* arXiv:2602.13171 (2026).
- G. Sidebottom. *A 55-Addition Rank-23 Scheme for* 3×3 *Matrix Multiplication via Exact Two-Sided Minimization.* Companion paper, this repository (2026). Artifacts: DOI `10.5281/zenodo.21240904`.
- L. de Moura, S. Ullrich. *The Lean 4 Theorem Prover and Programming Language.* CADE-28 (2021); and The Mathlib Community, *The Lean Mathematical Library*, CPP (2020).